

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



AAT REPORT
on

OPERATING SYSTEMS

Submitted by

Om G Gowdar (1WA24CS198)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING

in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by Om G Gowdar(1WA24CS198), who is bonafide student of B. M. S. College of Engineering. It is in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2024. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Associate Professor
Department of CSE
B. M. S. College of Engg, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
B. M. S. College of Engg, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS b) SJF c) Priority d) Round Robin (Experiment with different quantum sizes for RR algorithm)	
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one) a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	
4	Write a C program to simulate: (Any one) a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	
5	Write a C program to simulate: (Any one) a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	
6	Write a C program to simulate the following contiguous memory allocation techniques. (Any one) a) Worst-fit b) Best-fit c) First-fit	
7	Write a C program to simulate page replacement algorithms. (Any one) a) FIFO b) LRU c) Optimal	
8	Write a C program to simulate the following file allocation strategies. (Any one) a) Sequential b) Indexed c) Linked	

9	Write a C program to simulate the following file organization techniques. (Any one) a) Single level directory b) Two level directory c) Hierarchical	
10	Write a C program to simulate disk scheduling algorithms. (Any one) a) FCFS b) SCAN c) SSTF d) c-LOOK	

Course Outcomes:

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

1. Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one)

a) FCFS

b) SJF

c) Priority

d) Round Robin (Experiment with different quantum sizes for RR algorithm)

a)FCFS

```
#include <stdio.h>

int main()
{
    int n, i;

    printf("Enter total number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n];

    // Input
    for(i = 0; i < n; i++)
    {
        printf("\nProcess [%d]\n", i+1);

        printf("Arrival time: ");
        scanf("%d", &at[i]);

        printf("Burst time: ");
        scanf("%d", &bt[i]);
    }
```

```

int current_time = 0;

// FCFS Calculation
for(i = 0; i < n; i++)
{
    if(current_time < at[i])
        current_time = at[i];

    ct[i] = current_time + bt[i];
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    current_time = ct[i];
}

float avg_tat = 0, avg_wt = 0;

// Output Table
printf("\nProcess Arrival Time Burst Time Completion Time Turnaround Time Waiting Time\n");

for(i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);

    avg_tat += tat[i];
    avg_wt += wt[i];
}

avg_tat /= n;

```

```

    avg_wt /= n;

    printf("\nAverage Turnaround Time: %.2f", avg_tat);
    printf("\nAverage Waiting Time: %.2f\n", avg_wt);

    return 0;
}

```

```

Enter total number of processes: 4

Process [1]
Arrival time: 0
Burst time: 2

Process [2]
Arrival time: 1
Burst time: 2

Process [3]
Arrival time: 5
Burst time: 3

Process [4]
Arrival time: 6
Burst time: 4

Process Arrival Time Burst Time Completion Time Turnaround Time Waiting Time
1         0         2         2             2             0
2         1         2         4             3             1
3         5         3         8             3             0
4         6         4        12             6             2

Average Turnaround Time: 3.50
Average Waiting Time: 0.75

```

b)SJF Non Pre-emptive

```

#include <stdio.h>

int main()
{
    int n, i, j;

    printf("USN: 1WA24CS198");
}

```

```
printf("\nEnter number of processes: ");  
scanf("%d", &n);
```

```
int at[n], bt[n], ct[n], tat[n], wt[n];  
int finished[n];
```

```
for(i = 0; i < n; i++)  
    finished[i] = 0;
```

```
printf("Enter arrival times:\n");  
for(i = 0; i < n; i++)  
    scanf("%d", &at[i]);
```

```
printf("Enter burst times:\n");  
for(i = 0; i < n; i++)  
    scanf("%d", &bt[i]);
```

```
int current_time = 0, completed = 0;  
int min_index;
```

```
while(completed < n)  
{  
    min_index = -1;  
    int min_bt = 10000;  
  
    for(i = 0; i < n; i++)  
    {  
        if(at[i] <= current_time && finished[i] == 0)  
        {  
            if(bt[i] < min_bt)  
            {
```



```

        min_bt = bt[i];
        min_index = i;
    }
}

if(min_index == -1)
{
    current_time++;
}
else
{
    ct[min_index] = current_time + bt[min_index];
    tat[min_index] = ct[min_index] - at[min_index];
    wt[min_index] = tat[min_index] - bt[min_index];

    current_time = ct[min_index];
    finished[min_index] = 1;
    completed++;
}

float avg_tat = 0, avg_wt = 0;

printf("\nSJF scheduling:\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);

```

```

    avg_tat += tat[i];
    avg_wt += wt[i];
}

avg_tat /= n;
avg_wt /= n;

printf("\nAverage turnaround time:%fms\n", avg_tat);
printf("\nAverage waiting time:%fms\n", avg_wt);

return 0;
}

```

```

USN: 1WA24CS198
Enter number of processes: 4
Enter arrival times:
0
0
0
0
Enter burst times:
6
8
7
3

SJF scheduling:

```

PID	AT	BT	CT	TAT	WT
P1	0	6	9	9	3
P2	0	8	24	24	16
P3	0	7	16	16	9
P4	0	3	3	3	0

```

Average turnaround time:13.000000ms
Average waiting time:7.000000ms

```

SRTF

```
#include <stdio.h>

int main()
{
    int n, i, time = 0, completed = 0, min_index;

    printf("USN: 1WA24CS198");
    printf("\nEnter the number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n];

    for(i = 0; i < n; i++)
    {
        printf("Enter arrival time and burst time for process %d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i]; // remaining time
    }

    while(completed < n)
    {
        min_index = -1;
        int min_rt = 10000;

        for(i = 0; i < n; i++)
        {
            if(at[i] <= time && rt[i] > 0)
            {
                if(rt[i] < min_rt)
```

```

        {
            min_rt = rt[i];
            min_index = i;
        }
    }
}

if(min_index == -1)
{
    time++;
}
else
{
    rt[min_index]--;
    time++;

    if(rt[min_index] == 0)
    {
        completed++;
        ct[min_index] = time;
        tat[min_index] = ct[min_index] - at[min_index];
        wt[min_index] = tat[min_index] - bt[min_index];
    }
}
}

float avg_wt = 0, avg_tat = 0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)

```

```

{
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);

    avg_wt += wt[i];
    avg_tat += tat[i];
}

avg_wt /= n;
avg_tat /= n;

printf("\nAverage Waiting Time: %.2f\n", avg_wt);
printf("Average Turnaround Time: %.2f\n", avg_tat);

return 0;
}

```

```

USN: 1WA24CS198
Enter the number of processes: 4
Enter arrival time and burst time for process 1: 2
1
Enter arrival time and burst time for process 2: 1
5
Enter arrival time and burst time for process 3: 4
1
Enter arrival time and burst time for process 4: 0
6

PID    AT    BT    CT    TAT    WT
1       2     1     3     1     0
2       1     5     8     7     2
3       4     1     5     1     0
4       0     6     13    13     7

Average Waiting Time: 2.25
Average Turnaround Time: 5.50

```

Priority scheduling non-pre-emptive

```
#include <stdio.h>

int main()
{
    int n, i, current_time = 0, completed_count = 0;
    printf("USN:1WA24CS198");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], pr[n];
    int ct[n], tat[n], wt[n];
    int completed[n];

    // Input
    for(i = 0; i < n; i++)
    {
        printf("\nProcess %d\n", i + 1);

        printf("Process ID: ");
        scanf("%d", &pid[i]);

        printf("Arrival Time: ");
        scanf("%d", &at[i]);

        printf("Burst Time: ");
        scanf("%d", &bt[i]);

        printf("Priority: ");
        scanf("%d", &pr[i]);
    }
}
```

```

    completed[i] = 0;
}

while(completed_count < n)
{
    int highest_priority = 9999;
    int selected_process = -1;

    for(i = 0; i < n; i++)
    {
        if(at[i] <= current_time && completed[i] == 0)
        {
            if(pr[i] < highest_priority)
            {
                highest_priority = pr[i];
                selected_process = i;
            }
        }
    }

    if(selected_process == -1)
    {
        current_time++;
    }
    else
    {
        int start_time = current_time;

        ct[selected_process] = start_time + bt[selected_process];
    }
}

```

```

    tat[selected_process] =
        ct[selected_process] - at[selected_process];

    wt[selected_process] =
        tat[selected_process] - bt[selected_process];

    current_time = ct[selected_process];

    completed[selected_process] = 1;
    completed_count++;
}
}

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i], pr[i],
        ct[i], tat[i], wt[i]);
}

return 0;
}

```



```
USN:1WA24CS198
Enter number of processes: 5

Process 1
Process ID: 1
Arrival Time: 0
Burst Time: 3
Priority: 5

Process 2
Process ID: 2
Arrival Time: 2
Burst Time: 3
Priority: 3

Process 3
Process ID: 3
Arrival Time: 3
Burst Time: 5
Priority: 2

Process 4
Process ID: 4
Arrival Time: 4
Burst Time: 4
Priority: 4

Process 5
Process ID: 5
Arrival Time: 6
Burst Time: 1
Priority: 1

PID    AT    BT    PR    CT    TAT    WT
1       0     3     5     3     3     0
2       2     3     3    12    10     7
3       3     5     2     8     5     0
4       4     4     4    16    12     8
5       6     1     1     9     3     2

Process returned 0 (0x0)   execution time : 75.547 s
```

Priority scheduling Pre-emptive

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, i;
```

```
    printf("USN:1WA24CS198");
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int pid[n], at[n], bt[n], pr[n];
```

```
    int rt[n], ct[n], tat[n], wt[n];
```

```

// Input
for(i = 0; i < n; i++)
{
    printf("\nProcess %d\n", i + 1);

    printf("Process ID: ");
    scanf("%d", &pid[i]);

    printf("Arrival Time: ");
    scanf("%d", &at[i]);

    printf("Burst Time: ");
    scanf("%d", &bt[i]);

    printf("Priority: ");
    scanf("%d", &pr[i]);

    rt[i] = bt[i]; // Remaining time
}

int current_time = 0;
int completed = 0;

while(completed < n)
{
    int highest_priority = 9999;
    int selected_process = -1;

    // Find highest priority process
    for(i = 0; i < n; i++)
    {

```

```

if(at[i] <= current_time && rt[i] > 0)
{
    if(pr[i] < highest_priority)
    {
        highest_priority = pr[i];
        selected_process = i;
    }
}

// No process available
if(selected_process == -1)
{
    current_time++;
}
else
{
    rt[selected_process]--;
    current_time++;

    if(rt[selected_process] == 0)
    {
        ct[selected_process] = current_time;

        tat[selected_process] =
            ct[selected_process] - at[selected_process];

        wt[selected_process] =
            tat[selected_process] - bt[selected_process];

        completed++;
    }
}

```

```

    }
}
}

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i], pr[i],
        ct[i], tat[i], wt[i]);
}

return 0;
}

```

```

USN:1WA24CS198
Enter number of processes: 4

Process 1
Process ID: 1
Arrival Time: 0
Burst Time: 8
Priority: 3

Process 2
Process ID: 2
Arrival Time: 1
Burst Time: 2
Priority: 4

Process 3
Process ID: 3
Arrival Time: 5
Burst Time: 6
Priority: 2

Process 4
Process ID: 4
Arrival Time: 9
Burst Time: 1
Priority: 1

PID    AT    BT    PR    CT    TAT    WT
1       0     8     3     15    15     7
2       1     2     4     17    16    14
3       5     6     2     12     7     1
4       9     1     1     10     1     0

Process returned 0 (0x0)   execution time : 53.511 s

```

Round Robin

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, tq, i;
```

```
    printf("USN:1WA24CS198");
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter time quantum: ");
```

```
    scanf("%d", &tq);
```

```
    int pid[n], at[n], bt[n], rt[n];
```

```
    int wt[n], tat[n], ct[n];
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        printf("\nProcess %d\n", i + 1);
```

```
        printf("Process ID: ");
```

```
        scanf("%d", &pid[i]);
```

```
        printf("Arrival Time: ");
```

```
        scanf("%d", &at[i]);
```

```
        printf("Burst Time: ");
```

```
        scanf("%d", &bt[i]);
```

```
        rt[i] = bt[i];
```

```
}
```

```
int completed = 0;
```

```
int current_time = 0;
```

```
while(completed < n)
```

```
{
```

```
    int executed = 0;
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        if(at[i] <= current_time && rt[i] > 0)
```

```
        {
```

```
            executed = 1;
```

```
            if(rt[i] > tq)
```

```
            {
```

```
                current_time += tq;
```

```
                rt[i] -= tq;
```

```
            }
```

```
            else
```

```
            {
```

```
                current_time += rt[i];
```

```
                ct[i] = current_time;
```

```
                tat[i] = ct[i] - at[i];
```

```
                wt[i] = tat[i] - bt[i];
```

```
                rt[i] = 0;
```

```
                completed++;
```

```
            }
```

```

    }
}

if(executed == 0)
    current_time++;
}

float avg_wt = 0, avg_tat = 0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);

    avg_wt += wt[i];
    avg_tat += tat[i];
}

avg_wt /= n;
avg_tat /= n;

printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

return 0;
}

```

```

USN:1WA24CS198
Enter number of processes: 4
Enter time quantum: 2

Process 1
Process ID: 1
Arrival Time: 0
Burst Time: 3

Process 2
Process ID: 2
Arrival Time: 1
Burst Time: 1

Process 3
Process ID: 3
Arrival Time: 3
Burst Time: 2

Process 4
Process ID: 4
Arrival Time: 8
Burst Time: 4

PID    AT    BT    CT    TAT    WT
1       0     3     6     6     3
2       1     1     3     2     1
3       3     2     5     2     0
4       8     4    12     4     0

Average Waiting Time = 1.00
Average Turnaround Time = 3.50

Process returned 0 (0x0)   execution time : 102.549 s

```

2. Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>
```

```
#define MAX 20
```

```
struct Process {
    int pid, at, bt, ct, tat, wt, type;
```



```
};
```

```
int main() {
    struct Process p[MAX], sq[MAX], uq[MAX];
    int n, i, sq_count = 0, uq_count = 0, time = 0;

    printf("USN: 1WA24CS198");
    printf("\nEnter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);

        p[i].pid = i + 1;

        printf("Arrival Time: ");
        scanf("%d", &p[i].at);

        printf("Burst Time: ");
        scanf("%d", &p[i].bt);

        printf("Type (1 = System, 2 = User): ");
        scanf("%d", &p[i].type);

        if (p[i].type == 1)
            sq[sq_count++] = p[i];
        else
            uq[uq_count++] = p[i];
    }

    /* System Queue Execution */
    for (i = 0; i < sq_count; i++) {
        if (time < sq[i].at)
            time = sq[i].at;

        time += sq[i].bt;
        sq[i].ct = time;
    }
}
```

```
}
```

```
/* User Queue Execution */  
for (i = 0; i < uq_count; i++) {  
    if (time < uq[i].at)  
        time = uq[i].at;  
  
    time += uq[i].bt;  
    uq[i].ct = time;  
}
```

```
float total_tat = 0, total_wt = 0;
```

```
printf("\n--- System Queue ---\n");  
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
```

```
for (i = 0; i < sq_count; i++) {  
    sq[i].tat = sq[i].ct - sq[i].at;  
    sq[i].wt = sq[i].tat - sq[i].bt;  
  
    total_tat += sq[i].tat;  
    total_wt += sq[i].wt;  
  
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",  
        sq[i].pid, sq[i].at, sq[i].bt,  
        sq[i].ct, sq[i].tat, sq[i].wt);  
}
```

```
printf("\n--- User Queue ---\n");  
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
```

```
for (i = 0; i < uq_count; i++) {  
    uq[i].tat = uq[i].ct - uq[i].at;  
    uq[i].wt = uq[i].tat - uq[i].bt;  
  
    total_tat += uq[i].tat;  
    total_wt += uq[i].wt;
```

```

        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
               uq[i].pid, uq[i].at, uq[i].bt,
               uq[i].ct, uq[i].tat, uq[i].wt);
    }

    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

    return 0;
}

```

```

USN:1WA24CS198
Enter number of processes: 4

```

```

Process 1
Arrival Time: 0
Burst Time: 4
Type (1=System, 2=User): 1

```

```

Process 2
Arrival Time: 0
Burst Time: 3
Type (1=System, 2=User): 1

```

```

Process 3
Arrival Time: 0
Burst Time: 8
Type (1=System, 2=User): 2

```

```

Process 4
Arrival Time: 10
Burst Time: 5
Type (1=System, 2=User): 1

```

```

---System Queue---

```

PID	AT	BT	CT	TAT	WT
1	0	4	4	4	0
2	0	3	7	7	4
4	10	5	15	5	0

```

---User Queue---

```

PID	AT	BT	CT	TAT	WT
3	0	8	23	23	15

```

Average Turnaround Time = 9.75
Average Waiting Time = 4.75

```

3. Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one)

a) Rate- Monotonic

b) Earliest-deadline First

c) Proportional scheduling

Rate Monotonics

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#define MAX 10
```

```
int gcd(int a, int b) {  
    while (b != 0) {  
        int temp = b;  
        b = a % b;  
        a = temp;  
    }  
    return a;  
}
```

```
int lcm(int a, int b) {  
    return (a * b) / gcd(a, b);  
}
```

```
int find_lcm(int arr[], int n) {  
    int result = arr[0];  
    for (int i = 1; i < n; i++) {  
        result = lcm(result, arr[i]);  
    }  
    return result;  
}
```

```
int main() {  
    int n, i, j;  
    int burst[MAX], period[MAX];
```

```

int remaining[MAX];
int hyperperiod;
float utilization = 0.0, bound;

printf("USN:1WA24CS198");
printf("\nEnter the number of processes:");
scanf("%d", &n);

printf("Enter the CPU burst times:\n");
for (i = 0; i < n; i++) {
    scanf("%d", &burst[i]);
    remaining[i] = burst[i];
}

printf("Enter the time periods:\n");
for (i = 0; i < n; i++) {
    scanf("%d", &period[i]);
}

hyperperiod = find_lcm(period, n);
printf("LCM=%d\n\n", hyperperiod);

// Sort by period (priority)
for (i = 0; i < n - 1; i++) {
    for (j = i + 1; j < n; j++) {
        if (period[i] > period[j]) {
            int temp;

            temp = period[i];
            period[i] = period[j];
            period[j] = temp;

            temp = burst[i];
            burst[i] = burst[j];
            burst[j] = temp;

            temp = remaining[i];

```

```

        remaining[i] = remaining[j];
        remaining[j] = temp;
    }
}

```

```

printf("Rate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");

```

```

for (i = 0; i < n; i++) {
    printf("%d\t%d\t%d\n", i + 1, burst[i], period[i]);
    utilization += (float)burst[i] / period[i];
}

```

```

bound = n * (pow(2.0, 1.0 / n) - 1);

```

```

printf("\n%f <= %f ==> %s\n",
    utilization,
    bound,
    (utilization <= bound) ? "true" : "false");

```

```

printf("Scheduling occurs for %d ms\n\n", hyperperiod);

```

```

int time, current_task;

```

```

for (time = 0; time < hyperperiod; time++) {

```

```

    // Release tasks
    for (i = 0; i < n; i++) {
        if (time % period[i] == 0) {
            remaining[i] = burst[i];
        }
    }
}

```

```

current_task = -1;

```

```

// Pick highest priority task

```

```

for (i = 0; i < n; i++) {
    if (remaining[i] > 0) {
        current_task = i;
        break;
    }
}

if (current_task != -1) {
    printf("%dms onwards: Process %d running\n",
        time,
        current_task + 1);
    remaining[current_task]--;
} else {
    printf("%dms onwards: CPU is idle\n", time);
}

return 0;
}

```

```

USN:1WA24CS198
Enter the number of processes:2
Enter the CPU burst times:
20
35
Enter the time periods:
50
100
LCM=100

```

Rate Monotone Scheduling:

PID	Burst	Period
1	20	50
2	35	100

```

0.750000 <= 0.828427 ==> true
Scheduling occurs for 100 ms

```

```

0ms onwards: Process 1 running
1ms onwards: Process 1 running
2ms onwards: Process 1 running
3ms onwards: Process 1 running
4ms onwards: Process 1 running

```

EDF Scheduling

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int main() {
```

```
    int n;
```

```
    int burst[MAX], deadline[MAX], period[MAX];
```

```
    int remaining[MAX];
```

```
    int time = 0;
```

```
    printf("USN:1WA24CS198");
```

```
    printf("\nEnter the number of processes:");
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("Task %d (burst deadline period): ", i + 1);
```

```
        scanf("%d %d %d",
```

```
            &burst[i],
```

```
            &deadline[i],
```

```
            &period[i]);
```

```
        remaining[i] = 0;
```

```
    }
```

```
    printf("\nPID\tBurst\tDeadline\tPeriod\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d\t%d\t%d\t\t%d\n",
```

```
            i + 1,
```

```
            burst[i],
```

```
            deadline[i],
```

```
            period[i]);
```

```
    }
```

```
    int limit = 20;
```



```

printf("\nScheduling occurs for %d ms\n\n", limit);

while (time < limit) {

    for (int i = 0; i < n; i++) {
        if (time % period[i] == 0) {
            remaining[i] = burst[i];
        }
    }

    int min_deadline = 9999;
    int selected = -1;

    for (int i = 0; i < n; i++) {
        if (remaining[i] > 0) {
            if (deadline[i] < min_deadline) {
                min_deadline = deadline[i];
                selected = i;
            }
        }
    }

    if (selected == -1) {
        printf("%dms : CPU is idle.\n", time);
    } else {
        printf("%dms : Task %d is running.\n",
            time,
            selected + 1);
        remaining[selected]--;
    }

    time++;
}

return 0;
}

```

```

USN:1WA24CS198
Enter the number of processes:3
Task 1 (burst deadline period): 2 4 5
Task 2 (burst deadline period): 3 7 20
Task 3 (burst deadline period): 2 8 10

PID      Burst    Deadline    Period
1         2         4           5
2         3         7          20
3         2         8          10

Scheduling occurs for 20 ms

0ms : Task 1 is running.
1ms : Task 1 is running.
2ms : Task 2 is running.
3ms : Task 2 is running.
4ms : Task 2 is running.
5ms : Task 1 is running.
6ms : Task 1 is running.
7ms : Task 3 is running.
8ms : Task 3 is running.

```

Proportional Scheduling

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct {
    char name[5];
    int tickets;
} Process;

int main() {
    printf("USN:1WA24CS198\n\n");
    int n, total_tickets = 0;
    float total_T = 0.0;

    printf("Enter the number of Processes: ");

```

```

scanf("%d", &n);
Process p[n];
srand(time(NULL));

for (int i = 0; i < n; i++) {
    printf("\nProcess %d:\n", i + 1);
    sprintf(p[i].name, "P%d", i + 1);
    printf("Tickets: ");
    scanf("%d", &p[i].tickets);
    total_tickets += p[i].tickets;
    total_T += p[i].tickets;
}

printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;
scanf("%d", &m);

for (int i = 0; i < m; i++) {
    int winning_ticket = rand() % total_tickets + 1;
    int accumulated_tickets = 0;
    int winner_index;

    for (int j = 0; j < n; j++) {
        accumulated_tickets += p[j].tickets;
        if (winning_ticket <= accumulated_tickets) {
            winner_index = j;
            break;
        }
    }

    printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
        p[winner_index].name);
}

for (int i = 0; i < n; i++) {

```

```

printf("\nThe Process: %s gets %.2f%% of Processor Time.\n", p[i].name,
      ((p[i].tickets / total_T) * 100));
}

return 0;
}

```

```

USN:1WA24CS198

Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 40, Winner: P3
Tickets picked: 59, Winner: P3
Tickets picked: 3, Winner: P1
Tickets picked: 1, Winner: P1
Tickets picked: 14, Winner: P2

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.
○ omgowdar@0ms-MacBook-Air OS_AAT_1 % █

```

4. Write a C program to simulate: (Any one)

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

Producer-Consumer problem using semaphores.

```

#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#include<semaphore.h>

#define BUFFER_SIZE 5

```

```
#define MAX_ITEMS 10 // limit output
```

```
int buffer[BUFFER_SIZE];
```

```
int in = 0, out = 0;
```

```
sem_t empty, full;
```

```
pthread_mutex_t mutex;
```

```
int produced_count = 0;
```

```
int consumed_count = 0;
```

```
void* producer(void* arg) {
```

```
    int item;
```

```
    while (produced_count < MAX_ITEMS) {
```

```
        item = produced_count;
```

```
        sem_wait(&empty);
```

```
        pthread_mutex_lock(&mutex);
```

```
        buffer[in] = item;
```

```
        printf("Produced: %d at buffer[%d]\n", item, in);
```

```
        in = (in + 1) % BUFFER_SIZE;
```

```
        produced_count++;
```

```
        pthread_mutex_unlock(&mutex);
```

```
        sem_post(&full);
```

```
        sleep(1);
```

```
    }
```

```
    return NULL;
```

```
}
```

```
void* consumer(void* arg) {
```

```
    int item;
```

```

while (consumed_count < MAX_ITEMS) {
    sem_wait(&full);
    pthread_mutex_lock(&mutex);

    item = buffer[out];
    printf("Consumed: %d from buffer[%d]\n", item, out);

    out = (out + 1) % BUFFER_SIZE;
    consumed_count++;

    pthread_mutex_unlock(&mutex);
    sem_post(&empty);

    sleep(1);
}

return NULL;
}

int main() {
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
}

```

```
    return 0;  
}
```

```
Produced: 0 at buffer[0]  
Consumed: 0 from buffer[0]  
Produced: 1 at buffer[1]  
Consumed: 1 from buffer[1]  
Produced: 2 at buffer[2]  
Consumed: 2 from buffer[2]  
Produced: 3 at buffer[3]  
Consumed: 3 from buffer[3]  
Produced: 4 at buffer[4]  
Consumed: 4 from buffer[4]  
Produced: 5 at buffer[0]  
Consumed: 5 from buffer[0]  
Produced: 6 at buffer[1]  
Consumed: 6 from buffer[1]  
Produced: 7 at buffer[2]  
Consumed: 7 from buffer[2]  
Produced: 8 at buffer[3]  
Consumed: 8 from buffer[3]  
Produced: 9 at buffer[4]  
Consumed: 9 from buffer[4]  
  
=== Code Execution Successful ===
```

Dining-Philosopher's problem

```
#include<stdio.h>  
#include<pthread.h>  
#include<unistd.h>
```

```
#define N 5
```

```
pthread_mutex_t forks[N];  
pthread_t philosophers[N];
```

```
void* philosopher(void* num) {  
    int id = *(int*)num;
```

```
    int left = id;  
    int right = (id + 1) % N;
```

```
    while (1) {  
        printf("Philosopher %d is thinking.\n", id);  
        sleep(1);
```

```
        // Deadlock avoidance strategy
```

```
        if (id % 2 == 0) {  
            pthread_mutex_lock(&forks[left]);  
            printf("Philosopher %d picked up left fork %d.\n", id, left);
```

```
            pthread_mutex_lock(&forks[right]);  
            printf("Philosopher %d picked up right fork %d.\n", id, right);
```

```
        } else {  
            pthread_mutex_lock(&forks[right]);  
            printf("Philosopher %d picked up right fork %d.\n", id, right);
```

```
            pthread_mutex_lock(&forks[left]);  
            printf("Philosopher %d picked up left fork %d.\n", id, left);  
        }
```

```
        printf("Philosopher %d is eating.\n", id);  
        sleep(2);
```

```
        pthread_mutex_unlock(&forks[left]);  
        pthread_mutex_unlock(&forks[right]);
```

```
        printf("Philosopher %d put down forks %d and %d.\n",
```



```

        id, left, right);
    }

    return NULL;
}

int main() {
    int i;
    int ids[N];

    // Initialize forks
    for (i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    // Create philosopher threads
    for (i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL,
            philosopher, &ids[i]);
    }

    // Join threads
    for (i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }

    // Destroy mutexes (not reached)
    for (i = 0; i < N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}

```

Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 3 is thinking.
Philosopher 2 is thinking.
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 1 picked up right fork 2.
Philosopher 4 picked up left fork 4.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 0 picked up left fork 0.
Philosopher 3 picked up right fork 4.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 4 put down forks 4 and 0.

5. Write a C program to simulate: (Any one)

a) Bankers' algorithm for the purpose of deadlock avoidance.

b) Deadlock Detection

Bankers' algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, m, i, j, k;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    int allocation[n][m], max[n][m], need[n][m];
```

```
    int available[m], work[m];
```

```
    int finish[n], safeSequence[n];
```

```
    // Input Allocation Matrix
```

```
    printf("Enter Allocation Matrix:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        for(j = 0; j < m; j++)
```

```
        {
```

```
            scanf("%d", &allocation[i][j]);
```

```
        }
```

```
    }
```

```
    // Input Maximum Matrix
```

```
    printf("Enter Maximum Demand Matrix:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
    for(j = 0; j < m; j++)
    {
        scanf("%d", &max[i][j]);
    }
}
```

```
// Input Available Resources
printf("Enter Available Resources:\n");
for(i = 0; i < m; i++)
{
    scanf("%d", &available[i]);
    work[i] = available[i];
}
```

```
// Calculate Need Matrix
for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - allocation[i][j];
    }
}
```

```
// Initialize Finish Array
for(i = 0; i < n; i++)
{
    finish[i] = 0;
}
```

```
int count = 0;
```

```
// Safety Algorithm
while(count < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
```

```

{
    // Check if process is unfinished
    if(finish[i] == 0)
    {
        int possible = 1;

        // Check if Need <= Work
        for(j = 0; j < m; j++)
        {
            if(need[i][j] > work[j])
            {
                possible = 0;
                break;
            }
        }

        // If process can execute
        if(possible)
        {
            // Work = Work + Allocation
            for(k = 0; k < m; k++)
            {
                work[k] += allocation[i][k];
            }

            safeSequence[count] = i;
            count++;
            finish[i] = 1;
            found = 1;
        }
    }
}

// If no process found
if(found == 0)
{
    break;
}

```

```

    }
}

// Check Safe State
int safe = 1;

for(i = 0; i < n; i++)
{
    if(finish[i] == 0)
    {
        safe = 0;
        break;
    }
}

if(safe)
{
    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d", safeSequence[i]);

        if(i != n - 1)
        {
            printf(" -> ");
        }
    }
}
else
{
    printf("\nSystem is NOT in a safe state.\n");
}

return 0;
}

```

```
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
```

Deadlock Detection

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, m, i, j, k;
```

```
    printf("Enter the number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the number of resources: ");
```

```
    scanf("%d", &m);
```

```
    int allocation[n][m], request[n][m];
```

```
    int available[m], work[m];
```

```

int finish[n], safeSequence[n];

// Input Allocation Matrix
printf("Enter the allocation matrix:\n");
for(i = 0; i < n; i++)
{
    printf("Process %d: ", i);

    for(j = 0; j < m; j++)
    {
        scanf("%d", &allocation[i][j]);
    }
}

// Input Request Matrix
printf("Enter the request matrix:\n");
for(i = 0; i < n; i++)
{
    printf("Process %d: ", i);

    for(j = 0; j < m; j++)
    {
        scanf("%d", &request[i][j]);
    }
}

// Input Available Resources
printf("Enter the available resources: ");

for(i = 0; i < m; i++)
{
    scanf("%d", &available[i]);
    work[i] = available[i];
}

// Initialize Finish Array
for(i = 0; i < n; i++)

```



```

{
    finish[i] = 0;
}

int count = 0;

// Deadlock Detection Algorithm
while(count < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
    {
        // Process not finished
        if(finish[i] == 0)
        {
            int possible = 1;

            // Check if Request <= Work
            for(j = 0; j < m; j++)
            {
                if(request[i][j] > work[j])
                {
                    possible = 0;
                    break;
                }
            }

            // If request can be satisfied
            if(possible)
            {
                // Work = Work + Allocation
                for(k = 0; k < m; k++)
                {
                    work[k] += allocation[i][k];
                }
            }
        }
    }

    count++;
}

```

```

        finish[i] = 1;
        safeSequence[count] = i;
        count++;
        found = 1;
    }
}

// No process can proceed
if(found == 0)
{
    break;
}

// Check for deadlock
int deadlock = 0;

for(i = 0; i < n; i++)
{
    if(finish[i] == 0)
    {
        deadlock = 1;
        break;
    }
}

if(deadlock)
{
    printf("\nDeadlock detected among processes:\n");

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            printf("P%d ", i);
        }
    }
}

```

```

    }
}
else
{
    printf("\nSystem is in safe state.\n");
    printf("Safe Sequence is: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d ", safeSequence[i]);
    }
}

return 0;
}

```

The screenshot shows a Windows command prompt window with the following text:

```

C:\Users\Lakshitha.L\OneDrive X + v
Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter the request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter the available resources: 0 0 0
System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1

Process returned 0 (0x0)   execution time : 59.364 s
Press any key to continue.

```

Write a C program to simulate the following contiguous memory allocation techniques. (Any one)

a) Worst-fit

b) Best-fit

c) First-fit

```
#include <stdio.h>
```

```
void firstFit(int blocks[], int m, int process[], int n) {  
    int allocation[n], used[m];
```

```
    for(int i = 0; i < m; i++) used[i] = 0;
```

```
    for(int i = 0; i < n; i++) {  
        allocation[i] = -1;  
        for(int j = 0; j < m; j++) {  
            if(!used[j] && blocks[j] >= process[i]) {  
                allocation[i] = j;  
                used[j] = 1;  
                break;  
            }  
        }  
    }  
}
```

```
    printf("\n--- First Fit ---\n");  
    printf("Process No.\tProcess Size\tBlock No.\n");  
    for(int i = 0; i < n; i++) {  
        printf("%d\t\t%d\t\t", i + 1, process[i]);  
        if(allocation[i] != -1)  
            printf("%d\n", allocation[i] + 1);  
        else  
            printf("Not Allocated\n");  
    }  
}
```

```
void bestFit(int blocks[], int m, int process[], int n) {
```

```

int allocation[n], used[m];

for(int i = 0; i < m; i++) used[i] = 0;

for(int i = 0; i < n; i++) {
    allocation[i] = -1;
    int best = -1;

    for(int j = 0; j < m; j++) {
        if(!used[j] && blocks[j] >= process[i]) {
            if(best == -1 || blocks[j] < blocks[best])
                best = j;
        }
    }

    if(best != -1) {
        allocation[i] = best;
        used[best] = 1;
    }
}

printf("\n--- Best Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for(int i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t", i + 1, process[i]);
    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

void worstFit(int blocks[], int m, int process[], int n) {
    int allocation[n], used[m];

    for(int i = 0; i < m; i++) used[i] = 0;

```

```

for(int i = 0; i < n; i++) {
    allocation[i] = -1;
    int worst = -1;

    for(int j = 0; j < m; j++) {
        if(!used[j] && blocks[j] >= process[i]) {
            if(worst == -1 || blocks[j] > blocks[worst])
                worst = j;
        }
    }

    if(worst != -1) {
        allocation[i] = worst;
        used[worst] = 1;
    }
}

printf("\n--- Worst Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for(int i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t", i + 1, process[i]);
    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

int main() {
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int blocks[m];
    printf("Enter sizes of %d memory blocks:\n", m);
    for(int i = 0; i < m; i++)

```

```

        scanf("%d", &blocks[i]);

printf("Enter number of processes: ");
scanf("%d", &n);

int process[n];
printf("Enter sizes of %d processes:\n", n);
for(int i = 0; i < n; i++)
    scanf("%d", &process[i]);

firstFit(blocks, m, process, n);
bestFit(blocks, m, process, n);
worstFit(blocks, m, process, n);

return 0;
}

```

```

Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100
500
200
300
600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.    Process Size    Block No.
1              212          2
2              417          5
3              112          3
4              426        Not Allocated

--- Best Fit ---
Process No.    Process Size    Block No.
1              212          4
2              417          2
3              112          3
4              426          5

--- Worst Fit ---
Process No.    Process Size    Block No.
1              212          5
2              417          2
3              112          4
4              426        Not Allocated

```

Write a C program to simulate page replacement algorithms.

(Any one)

a) FIFO

b) LRU

c) Optimal

```
#include <stdio.h>
```

```
#define MAX 50
```

```
void printFrames(int f[], int n)
```

```
{
    printf("[");
    for (int i = 0; i < n; i++)
    {
        if (f[i] != -1)
            printf("%d", f[i]);
        else
            printf(" ");

        if (i != n - 1)
            printf(" ");
    }
    printf("]\n");
}
```

```
int search(int f[], int n, int p)
```

```
{
    for (int i = 0; i < n; i++)
        if (f[i] == p)
            return 1;
    return 0;
}
```

```
// FIFO
```



```

void FIFO(int p[], int n, int nf)
{
    int f[MAX], faults = 0, next = 0;

    for (int i = 0; i < nf; i++)
        f[i] = -1;

    printf("\n--- FIFO Page Replacement ---\n");

    for (int i = 0; i < n; i++)
    {
        if (!search(f, nf, p[i]))
        {
            f[next] = p[i];
            next = (next + 1) % nf;
            faults++;
        }

        printf("Page %d -> ", p[i]);
        printFrames(f, nf);
    }

    printf("Total Page Faults (FIFO): %d\n", faults);
}

```

// LRU

```

void LRU(int p[], int n, int nf)
{
    int f[MAX], time[MAX], faults = 0;

    for (int i = 0; i < nf; i++)
    {
        f[i] = -1;
        time[i] = -1;
    }

    printf("\n--- LRU Page Replacement ---\n");
}

```

```

for (int i = 0; i < n; i++)
{
    int found = 0, pos = 0;

    for (int j = 0; j < nf; j++)
    {
        if (f[j] == p[i])
        {
            found = 1;
            time[j] = i;
        }
    }

    if (!found)
    {
        for (int j = 0; j < nf; j++)
        {
            if (f[j] == -1)
            {
                pos = j;
                break;
            }

            if (time[j] < time[pos])
                pos = j;
        }

        f[pos] = p[i];
        time[pos] = i;
        faults++;
    }

    printf("Page %d -> ", p[i]);
    printFrames(f, nf);
}

```

```
    printf("Total Page Faults (LRU): %d\n", faults);  
}
```

```
// Optimal
```

```
void Optimal(int p[], int n, int nf)  
{
```

```
    int f[MAX], faults = 0;
```

```
    for (int i = 0; i < nf; i++)  
        f[i] = -1;
```

```
    printf("\n--- Optimal Page Replacement ---\n");
```

```
    for (int i = 0; i < n; i++)  
    {
```

```
        if (!search(f, nf, p[i]))  
        {
```

```
            int pos = -1, farthest = i;
```

```
            for (int j = 0; j < nf; j++)  
            {  
                int k;
```

```
                if (f[j] == -1)  
                {  
                    pos = j;  
                    break;  
                }
```

```
                for (k = i + 1; k < n; k++)  
                    if (f[j] == p[k])  
                        break;
```

```
                if (k > farthest)  
                {  
                    farthest = k;  
                    pos = j;
```

```

        }
    }

    f[pos] = p[i];
    faults++;
}

printf("Page %d -> ", p[i]);
printFrames(f, nf);
}

printf("Total Page Faults (Optimal): %d\n", faults);
}

int main()
{
    int n, nf, p[MAX];

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &p[i]);

    printf("Enter number of frames: ");
    scanf("%d", &nf);

    FIFO(p, n, nf);
    LRU(p, n, nf);
    Optimal(p, n, nf);

    return 0;
}

```

```

Om-1WA24CS198Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

--- FIFO Page Replacement ---
Page 7 -> [7  ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 3 1]
Page 0 -> [2 3 0]
Page 4 -> [4 3 0]
Page 2 -> [4 2 0]
Page 3 -> [4 2 3]
Page 0 -> [0 2 3]
Page 3 -> [0 2 3]
Total Page Faults (FIFO): 10

--- LRU Page Replacement ---
Page 7 -> [7  ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [4 0 3]
Page 2 -> [4 0 2]
Page 3 -> [4 3 2]
Page 0 -> [0 3 2]
Page 3 -> [0 3 2]
Total Page Faults (LRU): 9

--- Optimal Page Replacement ---
Page 7 -> [7  ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [2 4 3]
Page 2 -> [2 4 3]
Page 3 -> [2 4 3]
Page 0 -> [0 4 3]
Page 3 -> [0 4 3]
Total Page Faults (Optimal): 7

Process returned 0 (0x0)   execution time : 43.650 s

```

Write a C program to simulate the following file allocation strategies. (Any one)

- a) Sequential**
- b) Indexed**
- c) Linked**

Sequential

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int disk[50] = {0};
    int startBlock, len, i, j, flag;
    char choice;

    printf("--- Sequential File Allocation ---\n");

    do {
        flag = 0;
        printf("\nEnter starting block (0-49): ");
        scanf("%d", &startBlock);

        printf("Enter length of file: ");
        scanf("%d", &len);

        if (startBlock + len > 50) {
            printf("Error: File exceeds disk space.\n");
        } else {
            for (i = startBlock; i < startBlock + len; i++) {
                if (disk[i] == 1) {
                    flag = 1;
                    break;
                }
            }
        }

        if (flag == 1) {
            printf("Error: Contiguous blocks are not free.\n");
        } else {
            for (j = startBlock; j < startBlock + len; j++) {
                disk[j] = 1;
            }
        }
    }
```

```

        printf("File allocated successfully from block %d to %d.\n",
               startBlock, startBlock + len - 1);
    }
}

printf("Do you want to allocate another file? (y/n): ");
scanf(" %c", &choice);

} while (choice == 'y' || choice == 'Y');

return 0;
}

```

```

--- Sequential File Allocation ---

Enter starting block (0-49): 5
Enter length of file: 4
File allocated successfully from block 5 to 8.
Do you want to allocate another file? (y/n): y

Enter starting block (0-49): 6
Enter length of file: 2
Error: Contiguous blocks are not free.
Do you want to allocate another file? (y/n): n
omgowdar@Oms-MacBook-Air OS_AAT_1 % 

```

Indexed

```
#include <stdio.h>
```

```
int main() {
    int disk[50] = {0};
    int indexBlock, n, i, flag;
    int blocks[50];
    char choice;

    printf("--- Indexed File Allocation ---\n");

    do {
        printf("\nEnter index block (0-49): ");
        scanf("%d", &indexBlock);

        if (disk[indexBlock] == 1) {
            printf("Error: Index block %d is already allocated.\n",
indexBlock);
        } else {
            printf("Enter number of blocks for the file: ");
            scanf("%d", &n);

            printf("Enter the %d block numbers: ", n);

            flag = 0;

            for (i = 0; i < n; i++) {
                scanf("%d", &blocks[i]);

                if (disk[blocks[i]] == 1) {
                    flag = 1;
                }
            }

            if (flag == 1) {
                printf("Error: One or more blocks are already allocated.
Allocation failed.\n");
            }
        }
    } while (choice != 'q');
```



```

    } else {
        disk[indexBlock] = 1;

        for (i = 0; i < n; i++) {
            disk[blocks[i]] = 1;
        }

        printf("File allocated successfully.\n");
        printf("Index Block: %d\n", indexBlock);

        printf("Allocated Data Blocks: ");
        for (i = 0; i < n; i++) {
            printf("%d ", blocks[i]);
        }
        printf("\n");
    }
}

printf("Do you want to allocate another file? (y/n): ");
scanf(" %c", &choice);

} while (choice == 'y' || choice == 'Y');

return 0;
}

```

```

--- Indexed File Allocation ---

Enter index block (0-49): 10
Enter number of blocks for the file: 3
Enter the 3 block numbers: 15 22 24
File allocated successfully.
Index Block: 10
Allocated Data Blocks: 15 22 24
Do you want to allocate another file? (y/n): n
○ omgowdar@Oms-MacBook-Air OS_AAT_1 %

```

Linked

```
#include <stdio.h>
```

```
int main() {
    int disk[50] = {0};
    int n, blocks[50], i, flag;
    char choice;

    printf("--- Linked File Allocation ---\n");

    do {
        printf("\nEnter number of blocks for the file: ");
        scanf("%d", &n);

        printf("Enter %d block numbers: ", n);

        flag = 0;

        for (i = 0; i < n; i++) {
            scanf("%d", &blocks[i]);

            if (disk[blocks[i]] == 1) {
                flag = 1;
            }
        }

        if (flag == 1) {
            printf("Error: One or more blocks are already allocated.\n");
        } else {
            for (i = 0; i < n; i++) {
                disk[blocks[i]] = 1;
            }

            printf("File allocated using linked allocation.\n");
            printf("Linked Blocks: ");
        }
    } while (choice != 'q');
```

```

    for (i = 0; i < n; i++) {
        printf("%d", blocks[i]);

        if (i != n - 1)
            printf(" -> ");
    }
    printf(" -> NULL\n");
}

printf("Do you want to allocate another file? (y/n): ");
scanf(" %c", &choice);

} while (choice == 'y' || choice == 'Y');

return 0;
}

```

```

--- Linked File Allocation ---

Enter starting block (0-49): 3
Enter number of blocks required: 3
Enter the remaining 2 blocks to form the chain:
Next block after 3: 17
Next block after 17: 42
File allocated successfully. Chain: 3 -> 17 -> 42 -> NULL
Do you want to allocate another file? (y/n): n
○ omgowdar@Oms-MacBook-Air OS_AAT_1 %

```

Write a C program to simulate the following file organization techniques. (Any one)

a) Single level directory

b) Two level directory

c) Hierarchical

a) Single level directory

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char files[50][50];
```

```
    int count = 0, choice;
```

```
    char filename[50];
```

```
    do {
```

```
        printf("\n--- Single Level Directory ---\n");
```

```
        printf("1. Create File\n");
```

```
        printf("2. Display Files\n");
```

```
        printf("3. Search File\n");
```

```
        printf("4. Exit\n");
```

```
        printf("Enter choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch(choice) {
```

```
            case 1:
```

```
                printf("Enter file name: ");
```

```
                scanf("%s", files[count]);
```

```
                count++;
```

```
                break;
```

```
            case 2:
```

```
                printf("Files in Directory:\n");
```

```
                for(int i = 0; i < count; i++)
```

```
                    printf("%s\n", files[i]);
```

```
                break;
```

```

case 3:
    printf("Enter file name to search: ");
    scanf("%s", filename);

    for(int i = 0; i < count; i++) {
        if(strcmp(files[i], filename) == 0) {
            printf("File Found.\n");
            return 0;
        }
    }
    printf("File Not Found.\n");
    break;

case 4:
    break;

default:
    printf("Invalid Choice\n");
}

} while(choice != 4);

return 0;
}

```

```

Name: Om | USN: 1WA24CS198
Single Level Directory
Enter number of files: 2
Enter file name 1: photo.jpg
Enter file name 2: os_notes.pdf

Files in directory:
photo.jpg
os_notes.pdf
○ omgowdar@Oms-MacBook-Air OS_AAT_1 %

```

b) Two level directory

```
#include <stdio.h>
```

```
#include <string.h>
```

```
struct Directory {  
    char user[20];  
    char files[20][20];  
    int fileCount;  
};
```

```
int main() {  
    struct Directory dir[10];  
    int users, i, j;  
  
    printf("Enter number of users: ");  
    scanf("%d", &users);  
  
    for(i = 0; i < users; i++) {  
        printf("Enter username: ");  
        scanf("%s", dir[i].user);  
  
        printf("Enter number of files: ");  
        scanf("%d", &dir[i].fileCount);  
  
        for(j = 0; j < dir[i].fileCount; j++) {  
            printf("Enter file %d: ", j + 1);  
            scanf("%s", dir[i].files[j]);  
        }  
    }  
  
    printf("\nTwo Level Directory Structure:\n");  
  
    for(i = 0; i < users; i++) {  
        printf("\nUser: %s\n", dir[i].user);  
  
        for(j = 0; j < dir[i].fileCount; j++) {
```

```
        printf(" %s\n", dir[i].files[j]);
    }
}

return 0;
}
```

```
Name: Om | USN: 1WA24CS198
Two Level Directory
Enter number of users: 2

Enter username: Alice
Enter number of files for Alice: 2
Enter file 1: photo.jpg
Enter file 2: notes.pdf

Enter username: Bob
Enter number of files for Bob: 1
Enter file 1: notes.pdf

Directory Structure:

User: Alice
photo.jpg
notes.pdf

User: Bob
notes.pdf
○ omgowdar@Oms-MacBook-Air OS_AAT_1 %
```

c) Hierarchical

```
#include <stdio.h>
```

```
struct Directory {  
    char name[20];  
    int subdirs;  
};
```

```
int main() {  
    struct Directory root[20];  
    int n, i;  
  
    printf("Enter number of directories under root: ");  
    scanf("%d", &n);  
  
    for(i = 0; i < n; i++) {  
        printf("Enter directory name: ");  
        scanf("%s", root[i].name);  
  
        printf("Enter number of subdirectories in %s: ", root[i].name);  
        scanf("%d", &root[i].subdirs);  
    }  
  
    printf("\nHierarchical Directory Structure:\n");  
  
    for(i = 0; i < n; i++) {  
        printf("Root/%s (%d subdirectories)\n",  
            root[i].name,  
            root[i].subdirs);  
    }  
  
    return 0;  
}
```



```

Name: Om | USN: 1WA24CS198
Hierarchical Directory
Enter number of main directories: 1

Enter main directory name: Personal
Enter number of subdirectories in Personal: 2
Enter subdirectory name: 1
Enter number of files in 1: 2
Enter file 1: Photo.jpg
Enter file 2: notes.pdf
Enter subdirectory name: 2
Enter number of files in 2: 1
Enter file 1: Photo.jpg

Hierarchical Directory Structure:

Personal
  1
    Photo.jpg
    notes.pdf
  2
    Photo.jpg
○ omgowdar@Oms-MacBook-Air OS_AAT_1 %

```

Write a C program to simulate disk scheduling algorithms.

(Any one)

a) FCFS

b) SCAN

c) SSTF

d) c-LOOK

a) FCFS

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int req[100], n, head, seek = 0;
```

```
    printf("Enter number of requests: ");
```

```
    scanf("%d", &n);
```

```

printf("Enter request queue:\n");
for(int i = 0; i < n; i++)
    scanf("%d", &req[i]);

printf("Enter initial head position: ");
scanf("%d", &head);

printf("\nSeek Sequence: %d", head);

for(int i = 0; i < n; i++) {
    seek += abs(req[i] - head);
    head = req[i];
    printf(" -> %d", head);
}

printf("\nTotal Seek Time = %d\n", seek);

return 0;
}

```

```

Name: Om | USN: 1WA24CS198
FCFS Disk Scheduling
Enter number of requests: 5
Enter request queue:
23 45 140 19 170
Enter initial head position: 50

Seek Sequence:
50 -> 23 -> 45 -> 140 -> 19 -> 170

Total Seek Time = 416
○ omgowdar@Oms-MacBook-Air OS_AAT_1 % █

```

b) SCAN

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void sort(int a[], int n) {  
    for(int i = 0; i < n - 1; i++)  
        for(int j = 0; j < n - i - 1; j++)  
            if(a[j] > a[j + 1]) {  
                int t = a[j];  
                a[j] = a[j + 1];  
                a[j + 1] = t;  
            }  
}
```

```
int main() {  
    int req[100], n, head, seek = 0;  
  
    printf("Enter number of requests: ");  
    scanf("%d", &n);  
  
    printf("Enter request queue:\n");  
    for(int i = 0; i < n; i++)  
        scanf("%d", &req[i]);  
  
    printf("Enter initial head position: ");  
    scanf("%d", &head);  
  
    sort(req, n);  
  
    int pos;  
    for(pos = 0; pos < n; pos++)  
        if(req[pos] > head)  
            break;  
  
    printf("\nSeek Sequence: %d", head);
```

```

for(int i = pos; i < n; i++) {
    seek += abs(req[i] - head);
    head = req[i];
    printf(" -> %d", head);
}

for(int i = pos - 1; i >= 0; i--) {
    seek += abs(req[i] - head);
    head = req[i];
    printf(" -> %d", head);
}

printf("\nTotal Seek Time = %d\n", seek);

return 0;
}

```

```

Name: Om | USN: 1WA24CS198
SCAN Disk Scheduling
Enter number of requests: 5
Enter request queue:
23 45 170 140 19
Enter initial head position: 50
Enter disk size: 200
Enter direction (0 for left, 1 for right): 1

Seek Sequence:
50 -> 140 -> 170 -> 199 -> 45 -> 23 -> 19 ->

Total Seek Time = 329
omgowdar@Oms-MacBook-Air OS_AAT_1 %

```

c) SSTF

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int req[100], visited[100] = {0};
```

```
    int n, head, seek = 0;
```

```
    printf("Enter number of requests: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter request queue:\n");
```

```
    for(int i = 0; i < n; i++)
```

```
        scanf("%d", &req[i]);
```

```
    printf("Enter initial head position: ");
```

```
    scanf("%d", &head);
```

```
    printf("\nSeek Sequence: %d", head);
```

```
    for(int i = 0; i < n; i++) {
```

```
        int min = 9999, index = -1;
```

```
        for(int j = 0; j < n; j++) {
```

```
            if(!visited[j] && abs(req[j] - head) < min) {
```

```
                min = abs(req[j] - head);
```

```
                index = j;
```

```
            }
```

```
        }
```

```
        visited[index] = 1;
```

```
        seek += min;
```

```
        head = req[index];
```

```
        printf(" -> %d", head);
```

```
    }
```

```

printf("\nTotal Seek Time = %d\n", seek);

return 0;
}

```

```

Name: Om | USN: 1WA24CS198
SSTF Disk Scheduling
Enter number of requests: 5
Enter request queue:
23 45 170 140 19
Enter initial head position: 50

Seek Sequence:
45 -> 23 -> 19 -> 140 -> 170 ->

Total Seek Time = 182
○ omgowdar@Oms-MacBook-Air OS_AAT_1 %

```

d) c-LOOK

```

#include <stdio.h>
#include <stdlib.h>

void sort(int a[], int n) {
    for(int i = 0; i < n - 1; i++)
        for(int j = 0; j < n - i - 1; j++)
            if(a[j] > a[j + 1]) {
                int t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
            }
}

int main() {
    int req[100], n, head, seek = 0;

```

```

printf("Enter number of requests: ");
scanf("%d", &n);

printf("Enter request queue:\n");
for(int i = 0; i < n; i++)
    scanf("%d", &req[i]);

printf("Enter initial head position: ");
scanf("%d", &head);

sort(req, n);

int pos;
for(pos = 0; pos < n; pos++)
    if(req[pos] > head)
        break;

printf("\nSeek Sequence: %d", head);

for(int i = pos; i < n; i++) {
    seek += abs(req[i] - head);
    head = req[i];
    printf(" -> %d", head);
}

for(int i = 0; i < pos; i++) {
    seek += abs(req[i] - head);
    head = req[i];
    printf(" -> %d", head);
}

printf("\nTotal Seek Time = %d\n", seek);

return 0;
}

```

```
Name: Om | USN: 1WA24CS198  
C-LOOK Disk Scheduling  
Enter number of requests: 5  
Enter request queue:  
23 45 170 140 19  
Enter initial head position: 3
```

```
Seek Sequence:  
3 -> 19 -> 23 -> 45 -> 140 -> 170 ->
```

```
Total Seek Time = 167
```

```
○ omgowdar@Oms-MacBook-Air OS_AAT_1 % █
```